TROUBLESHOOTING

Troubleshoot installation and login

Fix command not found, PATH, permission, network, and authentication errors when installing or signing in to Claude Code.

If installation fails or you can't sign in, find your error below. For runtime issues after Claude Code is working, see **Troubleshooting**. For configuration problems such as settings not applying or hooks not firing, see **Debug your configuration**.

Find your error

Match the error message or symptom you're seeing to a fix:

What you see	Solution
`command not found: claude` or `'claude' is not recognized`	Fix your PATH
`syntax error near unexpected token '<'`	Install script returns HTML
`curl: (22) The requested URL returned error: 403`	Install script returned 403
`curl: (23)` or `curl: (56) Failure writing output to destination`	Check connectivity or use an alternative installer
`Killed` during install on Linux	Add swap space for low-memory servers
`TLS connect error` or `SSL/TLS secure channel`	Update CA certificates
`Failed to fetch version` or can't reach download server	Check network and proxy settings
`irm is not recognized` or `&& is not valid`	Use the right command for your shell
`Cask 'claude-code' is unavailable: No Cask with this name exists`	Update Homebrew
`'bash' is not recognized as the name of a cmdlet`	Use the Windows installer command
`Claude Code on Windows requires either Git for Windows (for bash) or PowerShell`	Install a shell
`Claude Code does not support 32-bit Windows`	Open Windows PowerShell, not the x86 entry

What you see	Solution
`The process cannot access the file ... because it is being used by another process`	**Clear the downloads folder and retry**
`Error loading shared library`	**Wrong binary variant for your system**
`Illegal instruction`	**Architecture or CPU instruction set mismatch**
`cannot execute binary file: Exec format error` in WSL	**WSL1 native-binary regression**
PowerShell installer completes but `claude` is not found or shows an old version	**Add the install directory to your PATH**, then open a new terminal
`dyld: cannot load`, `dyld: Symbol not found`, or `Abort trap` on macOS	**Binary incompatibility**
`Invoke-Expression: Missing argument in parameter list`	**Install script returns HTML**
`App unavailable in region`	Claude Code is not available in your country. See **supported countries**.
`unable to get local issuer certificate`	**Configure corporate CA certificates**
`OAuth error` or `403 Forbidden`	**Fix authentication**
`Could not load the default credentials` or `Could not load credentials from any providers`	**Bedrock, Vertex, or Foundry credentials**
`ChainedTokenCredential authentication failed` or `CredentialUnavailableError`	**Bedrock, Vertex, or Foundry credentials**
`API Error: 500`, `529 Overloaded`, `429`, or other 4xx and 5xx errors not listed above	See the **Error reference**

If your issue isn't listed, work through the diagnostic checks below to narrow down the cause.

> If you'd rather skip the terminal entirely, the **Claude Code Desktop app** lets you install and use Claude Code through a graphical interface. Download it for **macOS** or **Windows** and start coding without any command-line setup.

Run diagnostic checks

Check network connectivity

The installer downloads from `downloads.claude.ai`. Verify you can reach it:

```
curl -sI https://downloads.claude.ai/claude-code-releases/latest
```

In PowerShell, run `curl.exe -sI` instead. PowerShell aliases `curl` to `Invoke-WebRequest`, which rejects the `-sI` flags.

An `HTTP/2 200` line means you reached the server. If you see no output, `Could not resolve host`, or a connection timeout, your network is blocking the connection. Common causes:

- Corporate firewalls or proxies blocking `downloads.claude.ai`
- Regional network restrictions: try a VPN or alternative network
- TLS/SSL issues: update your system's CA certificates, or check if `HTTPS_PROXY` is configured

If you're behind a corporate proxy, set `HTTPS_PROXY` and `HTTP_PROXY` to your proxy's address before installing. Ask your IT team for the proxy URL if you don't know it, or check your browser's proxy settings.

This example sets both proxy variables, then runs the installer through your proxy:

macOS/Linux

```
export HTTP_PROXY=http://proxy.example.com:8080
export HTTPS_PROXY=http://proxy.example.com:8080
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell

```
$env:HTTP_PROXY = 'http://proxy.example.com:8080'
$env:HTTPS_PROXY = 'http://proxy.example.com:8080'
irm https://claude.ai/install.ps1 | iex
```

```
echo $PATH | tr ':' '\n' | grep -Fx "$HOME/.local/bin"
```

If this prints `/Users/you/.local/bin` or `/home/you/.local/bin`, the directory is in your PATH and you can skip to **Check for conflicting installations**. If there's no output, add it to your shell configuration.For Zsh, the default on macOS:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

For Bash, the default on most Linux distributions:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Alternatively, close and reopen your terminal.For other shells such as fish or Nushell, add `~/.local/bin` to your PATH using your shell's own configuration syntax, then restart your terminal.Verify the fix worked:

```
claude --version
```

```
$env:PATH -split ';' | Select-String '\.local\\bin'
```

If there's no output, add the install directory to your User PATH:

```
$currentPath = [Environment]::GetEnvironmentVariable('PATH',
'User')
[Environment]::SetEnvironmentVariable('PATH',
"$currentPath;$env:USERPROFILE\.local\bin", 'User')
```

Restart your terminal for the change to take effect.Verify the fix worked:

```
claude --version
```

```
echo %PATH% | findstr /i "local\bin"
```

If there's no output, open System Settings, go to Environment Variables, and add `%USERPROFILE%\.local\bin` to your User PATH variable. Restart your terminal.Verify the fix worked:

```
claude --version
```

List all `claude` binaries found in your PATH:

```
which -a claude
```

If this prints nothing, no `claude` is on your PATH yet. Go back to **Verify your PATH**.Check the three locations a `claude` binary can come from. `~/.local/bin/claude` is the native installer, `~/.claude/local/` is a legacy local npm install created by older versions of Claude Code, and the npm global list shows a `-g` install:

```
ls -la ~/.local/bin/claude
```

If either `ls` command prints `No such file or directory`, that's not an error. It means nothing is installed at that location, so move on to the next check.

```
ls -la ~/.claude/local/
```

```
npm -g ls @anthropic-ai/claude-code 2>/dev/null
```

List all `claude` binaries found in your PATH:

```
where.exe claude
```

Check whether the native installer placed a binary:

```
Test-Path "$env:USERPROFILE\.local\bin\claude.exe"
```

Verify your PATH

If installation succeeded but you get a `command not found` or `not recognized` error when running `claude`, the install directory isn't in your PATH. Your shell searches for programs in directories listed in PATH, and the installer places `claude` at `~/.local/bin/claude` on macOS/Linux or `%USERPROFILE%\.local\bin\claude.exe` on Windows.

> The **VS Code extension** does not place `claude` at this location. It bundles a private copy of the CLI inside the extension directory for its own chat panel and does not add it to PATH. If you have only installed the extension, `~/.local/bin/claude` will not exist. Run the **standalone install** to use `claude` from a terminal, then continue below.

Check if the install directory is in your PATH by listing your PATH entries and filtering for `local/bin`:

macOS/Linux

```
export HTTP_PROXY=http://proxy.example.com:8080
export HTTPS_PROXY=http://proxy.example.com:8080
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell

```
$env:HTTP_PROXY = 'http://proxy.example.com:8080'
$env:HTTPS_PROXY = 'http://proxy.example.com:8080'
irm https://claude.ai/install.ps1 | iex
```

Windows CMD

```
echo $PATH | tr ':' '\n' | grep -Fx "$HOME/.local/bin"
```

If this prints `/Users/you/.local/bin` or `/home/you/.local/bin`, the directory is in your PATH and you can skip to **Check for conflicting installations**. If there's no output, add it to your shell configuration.For Zsh, the default on macOS:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

For Bash, the default on most Linux distributions:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Alternatively, close and reopen your terminal.For other shells such as fish or Nushell, add `~/.local/bin` to your PATH using your shell's own configuration syntax, then restart your terminal.Verify the fix worked:

```
claude --version
```

```
$env:PATH -split ';' | Select-String '\.local\\bin'
```

If there's no output, add the install directory to your User PATH:

```
$currentPath = [Environment]::GetEnvironmentVariable('PATH',
'User')
[Environment]::SetEnvironmentVariable('PATH',
"$currentPath;$env:USERPROFILE\.local\bin", 'User')
```

Restart your terminal for the change to take effect.Verify the fix worked:

```
claude --version
```

```
echo %PATH% | findstr /i "local\bin"
```

If there's no output, open System Settings, go to Environment Variables, and add `%USERPROFILE%\.local\bin` to your User PATH variable. Restart your terminal.Verify the fix worked:

```
claude --version
```

List all `claude` binaries found in your PATH:

```
which -a claude
```

If this prints nothing, no `claude` is on your PATH yet. Go back to **Verify your PATH**.Check the three locations a `claude` binary can come from. `~/.local/bin/claude` is the native installer, `~/.claude/local/` is a legacy local npm install created by older versions of Claude Code, and the

npm global list shows a `-g` install:

```
ls -la ~/.local/bin/claude
```

If either `ls` command prints `No such file or directory`, that's not an error. It means nothing is installed at that location, so move on to the next check.

```
ls -la ~/.claude/local/
```

```
npm -g ls @anthropic-ai/claude-code 2>/dev/null
```

List all `claude` binaries found in your PATH:

```
where.exe claude
```

Check whether the native installer placed a binary:

```
Test-Path "$env:USERPROFILE\.local\bin\claude.exe"
```

Check for conflicting installations

Multiple Claude Code installations can cause version mismatches or unexpected behavior. Check what's installed:

macOS/Linux

```
export HTTP_PROXY=http://proxy.example.com:8080
export HTTPS_PROXY=http://proxy.example.com:8080
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell

```
$env:HTTP_PROXY = 'http://proxy.example.com:8080'
$env:HTTPS_PROXY = 'http://proxy.example.com:8080'
irm https://claude.ai/install.ps1 | iex
```

```
echo $PATH | tr ':' '\n' | grep -Fx "$HOME/.local/bin"
```

If this prints `/Users/you/.local/bin` or `/home/you/.local/bin`, the directory is in your PATH and you can skip to **Check for conflicting installations**. If there's no output, add it to your shell configuration.For Zsh, the default on macOS:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

For Bash, the default on most Linux distributions:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Alternatively, close and reopen your terminal.For other shells such as fish or Nushell, add `~/.local/bin` to your PATH using your shell's own configuration syntax, then restart your terminal.Verify the fix worked:

```
claude --version
```

```
$env:PATH -split ';' | Select-String '\.local\\bin'
```

If there's no output, add the install directory to your User PATH:

```
$currentPath = [Environment]::GetEnvironmentVariable('PATH',
'User')
[Environment]::SetEnvironmentVariable('PATH',
"$currentPath;$env:USERPROFILE\.local\bin", 'User')
```

Restart your terminal for the change to take effect.Verify the fix worked:

```
claude --version
```

```
echo %PATH% | findstr /i "local\bin"
```

If there's no output, open System Settings, go to Environment Variables, and add `%USERPROFILE%\.local\bin` to your User PATH variable. Restart your terminal.Verify the fix worked:

```
claude --version
```

List all `claude` binaries found in your PATH:

```
which -a claude
```

If this prints nothing, no `claude` is on your PATH yet. Go back to **Verify your PATH**.Check the three locations a `claude` binary can come from. `~/.local/bin/claude` is the native installer, `~/.claude/local/` is a legacy local npm install created by older versions of Claude Code, and the npm global list shows a `-g` install:

```
ls -la ~/.local/bin/claude
```

If either `ls` command prints `No such file or directory`, that's not an error. It means nothing is installed at that location, so move on to the next check.

```
ls -la ~/.claude/local/
```

```
npm -g ls @anthropic-ai/claude-code 2>/dev/null
```

List all `claude` binaries found in your PATH:

```
where.exe claude
```

Check whether the native installer placed a binary:

```
Test-Path "$env:USERPROFILE\.local\bin\claude.exe"
```

If you find multiple installations, keep only one. The native install at `~/.local/bin/claude` on macOS/Linux or `%USERPROFILE%\.local\bin\claude.exe` on Windows is recommended. Remove the extras:

Uninstall an npm global install:

```
npm uninstall -g @anthropic-ai/claude-code
```

Remove the legacy local npm install:

```
rm -rf ~/.claude/local
```

On Windows, use PowerShell:

```
Remove-Item -Recurse -Force "$env:USERPROFILE\.claude\local"
```

Remove a Homebrew install on macOS. If you installed the `claude-code@latest` cask, substitute that name:

```
brew uninstall --cask claude-code
```

Remove a WinGet install on Windows:

```
winget uninstall Anthropic.ClaudeCode
```

Check directory permissions

The installer needs write access to `~/.local/bin/` and `~/.claude/` on macOS and Linux. On Windows the install location is under `%USERPROFILE%`, which is writable by your user by default, so this section rarely applies there.

Check whether the directories are writable:

```
test -w ~/.local/bin && echo "writable" || echo "not writable"
test -w ~/.claude && echo "writable" || echo "not writable"
```

If either directory isn't writable, create the install directory and set your user as the owner:

```
sudo mkdir -p ~/.local/bin
sudo chown -R $(whoami) ~/.local
```

Verify the binary works

If `claude --version` prints a version but `claude` crashes or hangs on startup, run these checks to narrow down the cause. If `claude --version` says command not found, go to **Verify your PATH** first; the commands below assume `claude` is on your PATH.

Confirm the binary exists and is executable:

```
ls -la "$(command -v claude)"
```

On Windows, use PowerShell:

```
Get-Command claude | Select-Object Source
```

On Linux, check for missing shared libraries. If `ldd` shows missing libraries, you may need to install system packages. On Alpine Linux and other musl-based distributions, see **Alpine Linux setup**.

```
ldd "$(command -v claude)" | grep "not found"
```

Confirm the binary can execute:

```
claude --version
```

Common installation issues

These are the most frequently encountered installation problems and their solutions.

Install script returns HTML instead of a shell script

When running the install command, you may see one of these errors:

```
bash: line 1: syntax error near unexpected token `<'
bash: line 1: `<!DOCTYPE html>'
```

On PowerShell, the same problem appears as:

```
Invoke-Expression: Missing argument in parameter list.
```

Depending on how the request was routed, you may instead see a 403 with no HTML body:

```
curl: (22) The requested URL returned error: 403
```

These all mean the install URL returned an HTML page or an error status instead of the install script. If the HTML page says “App unavailable in region,” Claude Code is not available in your country. See **supported countries**.

A bare 403 with no body often has the same cause, but it can also come from a corporate proxy or firewall blocking the download. If you are in a supported country and still see the 403, work through **Check network connectivity** before trying the alternative installers below, since those reach the same hosts.

Otherwise, this can happen due to network issues, regional routing, or a temporary service disruption.

Solutions:

1. **Use an alternative install method**:

   On macOS, install via Homebrew:

   ```
   brew install --cask claude-code
   ```

   On Windows, install via WinGet:

   ```
   winget install Anthropic.ClaudeCode
   ```

2. **Retry after a few minutes**: the issue is often temporary. Wait and try the original command

again.

`command not found: claude` after installation

The install finished but `claude` doesn't work. The exact error varies by platform:

Platform	Error message
macOS	`zsh: command not found: claude`
Linux	`bash: claude: command not found`
Windows CMD	`'claude' is not recognized as an internal or external command`
PowerShell	`claude : The term 'claude' is not recognized as the name of a cmdlet`

This means the install directory isn't in your shell's search path. See **Verify your PATH** for the fix on each platform.

`curl: (56) Failure writing output to destination`

The `curl ... | bash` command downloads the script and pipes it to Bash for execution. This error, and the related `curl: (23) Failure writing output to destination`, means Bash did not receive the complete script. Exit code 56 indicates the download itself was interrupted, and exit code 23 indicates curl could not write what it received to the pipe, usually because Bash exited early.

Solutions:

1. **Check network stability**: Claude Code binaries are hosted at `downloads.claude.ai`. Test that you can reach it:

   ```
   curl -sI https://downloads.claude.ai/claude-code-releases/latest
   ```

   An `HTTP/2 200` line means you reached the server and the original failure was likely intermittent; retry the install command. If you see `Could not resolve host` or a connection timeout, your network is blocking the download.

2. **Try an alternative install method**:

   On macOS:

```
brew install --cask claude-code
```

On Windows:

```
winget install Anthropic.ClaudeCode
```

Homebrew cask unavailable or outdated

Homebrew reports `Error: Cask 'claude-code' is unavailable: No Cask with this name exists` when your local copy of the Homebrew cask index predates the cask's publication. Refresh the index and retry:

```
brew update
brew install --cask claude-code
```

If Homebrew installs an older Claude Code version than you expect, the same stale index is usually the cause. The `claude-code` cask tracks the stable channel and is typically about one week behind the latest release; for the newest version run `brew install --cask claude-code@latest` instead. See **Configure release channel** for the difference between the two casks.

TLS or SSL connection errors

Errors like `curl: (35) TLS connect error`, `schannel: next InitializeSecurityContext failed`, or PowerShell's `Could not establish trust relationship for the SSL/TLS secure channel` indicate TLS handshake failures.

Solutions:

1. **Update your system CA certificates**:

   On Ubuntu/Debian:

   ```
   sudo apt-get update && sudo apt-get install ca-certificates
   ```

   On macOS, the system curl uses the Keychain trust store; updating macOS itself updates the root certificates.

2. **On Windows, enable TLS 1.2** in PowerShell before running the installer:

```
[Net.ServicePointManager]::SecurityProtocol =
[Net.SecurityProtocolType]::Tls12
irm https://claude.ai/install.ps1 | iex
```

3. **Check for proxy or firewall interference**: corporate proxies that perform TLS inspection can cause these errors, including `unable to get local issuer certificate` and `SELF_SIGNED_CERT_IN_CHAIN`. For the install step, point curl at your corporate CA bundle with `--cacert`:

```
curl --cacert /path/to/corporate-ca.pem -fsSL
https://claude.ai/install.sh | bash
```

   For Claude Code itself once installed, set `NODE_EXTRA_CA_CERTS` so API requests trust the same bundle:

```
export NODE_EXTRA_CA_CERTS=/path/to/corporate-ca.pem
```

   Ask your IT team for the certificate file if you don't have it. You can also try on a direct connection to confirm the proxy is the cause.

4. **On Windows, bypass certificate revocation checks** if you see `CRYPT_E_NO_REVOCATION_CHECK (0x80092012)` or `CRYPT_E_REVOCATION_OFFLINE (0x80092013)`. These mean curl reached the server but your network blocks the certificate revocation lookup, which is common behind corporate firewalls. Add `--ssl-revoke-best-effort` to the install command:

```
curl --ssl-revoke-best-effort -fsSL
https://claude.ai/install.cmd -o install.cmd && install.cmd
&& del install.cmd
```

   Alternatively, install with `winget install Anthropic.ClaudeCode`, which avoids curl entirely.

`Failed to fetch version from downloads.claude.ai`

The installer couldn't reach the download server. This typically means `downloads.claude.ai` is

blocked on your network.

Solutions:

1. **Test connectivity directly**:

```
curl -sI https://downloads.claude.ai/claude-code-
releases/latest
```

2. **If behind a proxy**, set `HTTPS_PROXY` so the installer can route through it. See **proxy configuration** for details.

```
export HTTPS_PROXY=http://proxy.example.com:8080
curl -fsSL https://claude.ai/install.sh | bash
```

3. **If on a restricted network**, try a different network or VPN, or use an alternative install method:

   On macOS:

```
brew install --cask claude-code
```

   On Windows:

```
winget install Anthropic.ClaudeCode
```

Wrong install command on Windows

If you see `'irm' is not recognized`, `The token '&&' is not valid`, or `'bash' is not recognized as the name of a cmdlet`, you copied the install command for a different shell or operating system.

- `irm` **not recognized**: you're in CMD, not PowerShell. You have two options:

  Open PowerShell by searching for "PowerShell" in the Start menu, then run the original install command:

```
irm https://claude.ai/install.ps1 | iex
```

Or stay in CMD and use the CMD installer instead:

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd &&
install.cmd && del install.cmd
```

- `&&` **not valid**: you're in PowerShell but ran the CMD installer command. Use the PowerShell installer:

```
irm https://claude.ai/install.ps1 | iex
```

- `bash` **not recognized**: you ran the macOS/Linux installer on Windows. Use the PowerShell installer instead:

```
irm https://claude.ai/install.ps1 | iex
```

`The process cannot access the file` during Windows install

If the PowerShell installer fails with `Failed to download binary: The process cannot access the file ... because it is being used by another process`, the installer couldn't write to `%USERPROFILE%\.claude\downloads`. This usually means a previous install attempt is still running, or antivirus software is scanning a partially downloaded binary in that folder.

Close any other PowerShell windows running the installer and wait for antivirus scans to release the file. Then delete the downloads folder and run the installer again:

```
Remove-Item -Recurse -Force "$env:USERPROFILE\.claude\downloads"
irm https://claude.ai/install.ps1 | iex
```

Install killed on low-memory Linux servers

If you see `Killed` during installation on a VPS or cloud instance:

```
Setting up Claude Code...
Installing Claude Code native build latest...
bash: line 142: 34803 Killed    "$binary_path" install
${TARGET:+"$TARGET"}
```

The Linux OOM killer terminated the process because the system ran out of memory. Claude Code requires at least 4 GB of available RAM.

Solutions:

1. **Add swap space** if your server has limited RAM. Swap uses disk space as overflow memory, letting the install complete even with low physical RAM.

   Create a 2 GB swap file and enable it:

   ```
   sudo fallocate -l 2G /swapfile
   sudo chmod 600 /swapfile
   sudo mkswap /swapfile
   sudo swapon /swapfile
   ```

   Then retry the installation:

   ```
   curl -fsSL https://claude.ai/install.sh | bash
   ```

2. **Close other processes** to free memory before installing.
3. **Use a larger instance** if possible. Claude Code requires at least 4 GB of RAM.

Install hangs in Docker

When installing Claude Code in a Docker container, installing as root into `/` can cause hangs.

Solutions:

1. **Set a working directory** before running the installer. When run from `/`, the installer scans the entire filesystem, which causes excessive memory usage. Setting `WORKDIR` limits the scan to a small directory:

   ```
   WORKDIR /tmp
   RUN curl -fsSL https://claude.ai/install.sh | bash
   ```

2. **Increase Docker memory limits** if using Docker Desktop:

```
docker build --memory=4g .
```

Claude Desktop overrides the `claude` command on Windows

If you installed an older version of Claude Desktop, it may register a `Claude.exe` in the `WindowsApps` directory that takes PATH priority over Claude Code CLI. Running `claude` opens the Desktop app instead of the CLI.

Update Claude Desktop to the latest version to fix this issue.

Claude Code on Windows requires either Git for Windows (for bash) or PowerShell

Git for Windows is optional. Claude Code uses the **PowerShell tool** when Git Bash is absent, so this error means neither shell was found.

If PowerShell is missing from your PATH, its default location is `C:\Windows\System32\WindowsPowerShell\v1.0\`. Add that directory to your `PATH`, or install **PowerShell 7**, which provides `pwsh`.

To install Git for Windows instead, download it from **git-scm.com/downloads/win**. During setup, select "Add to PATH." Restart your terminal after installing. Installing it enables the Bash tool, useful when working with Bash-based scripts and tooling.

If Git is already installed but Claude Code can't find it, set the path in your **settings.json file**:

```
{
  "env": {
    "CLAUDE_CODE_GIT_BASH_PATH": "C:\\Program
Files\\Git\\bin\\bash.exe"
  }
}
```

If your Git is installed somewhere else, find the path by running `where.exe git` in PowerShell and use the `bin\bash.exe` path from that directory.

If the path is correct and the file exists but Claude Code still reports it as not found, endpoint security software such as AppLocker, Group Policy software restriction policies, or EDR agents may

be interfering. On versions before v2.1.116, Claude Code spawned a child process (`cmd.exe`) to verify the path, which these policies can block — a common signal is that `cmd.exe /c dir "C:\Program Files\Git\bin\bash.exe"` works when you run it directly in PowerShell but fails silently when launched by `claude.exe` .

Claude Code v2.1.116 and later check the filesystem directly, so update first. If the error persists on a current version, ask your IT team to allowlist `claude.exe` and the processes it spawns, including `cmd.exe` and `bash.exe` , in your endpoint protection policy.

Claude Code does not support 32-bit Windows

Windows includes two PowerShell entries in the Start menu: `Windows PowerShell` and `Windows PowerShell (x86)` . The x86 entry runs as a 32-bit process and triggers this error even on a 64-bit machine. To check which case you're in, run this in the same window that produced the error:

```
[Environment]::Is64BitOperatingSystem
```

If this prints `True` , your operating system is fine. Close the window, open `Windows PowerShell` without the x86 suffix, and run the install command again.

If this prints `False` , you are on a 32-bit edition of Windows. Claude Code requires a 64-bit operating system. See the **system requirements**.

Linux musl or glibc binary mismatch

If you see errors about missing shared libraries like `libstdc++.so.6` or `libgcc_s.so.1` after installation, the installer may have downloaded the wrong binary variant for your system.

```
Error loading shared library libstdc++.so.6: No such file or
directory
```

This can happen on glibc-based systems that have musl cross-compilation packages installed, causing the installer to misdetect the system as musl.

Solutions:

1. **Check which libc your system uses**:

```
ldd --version 2>&1 | head -1
```

Output mentioning `GNU libc` or `GLIBC` means glibc. Output mentioning `musl` means musl.

2. **If you're on glibc but got the musl binary**, remove the installation and reinstall. You can also manually download the correct binary using the manifest at `https://downloads.claude.ai/claude-code-releases/{VERSION}/manifest.json`. File a **GitHub issue** with the output of `ldd --version` and `ls /lib/libc.musl*`.
3. **If you're actually on musl**, such as Alpine Linux, install the required packages:

```
apk add libgcc libstdc++ ripgrep
```

`Illegal instruction`

If running `claude` or the installer prints `Illegal instruction`, the native binary uses CPU instructions your processor doesn't support. There are two distinct causes.

Architecture mismatch. The installer downloaded the wrong binary, for example x86 on an ARM server. Check with `uname -m` on macOS or Linux, or `$env:PROCESSOR_ARCHITECTURE` in PowerShell. If the result doesn't match the binary you received, **file a GitHub issue** with the output.

Missing AVX instruction set. If your architecture is correct but you still see `Illegal instruction`, your CPU likely lacks AVX or another instruction the binary requires. This affects roughly pre-2013 Intel and AMD processors, and virtual machines where the hypervisor does not pass AVX through to the guest.

On a VPS or VM, run `grep -m1 -ow avx /proc/cpuinfo`; an empty result means AVX is not available to the guest.

There is no native-binary workaround; track **issue #50384** for status, and include your CPU model from `grep -m1 "model name" /proc/cpuinfo` on Linux or `sysctl -n machdep.cpu.brand_string` on macOS when reporting.

Alternative install methods download the same native binary and won't resolve either cause.

`dyld: cannot load` on macOS

If you see `dyld: cannot load`, `dyld: Symbol not found`, or `Abort trap: 6` during installation,

the binary is incompatible with your macOS version or hardware.

```
dyld: cannot load 'claude-2.1.42-darwin-x64' (load command
0x80000034 is unknown)
Abort trap: 6
```

A `Symbol not found` error that references `libicucore` also indicates your macOS version is older than the binary supports:

```
dyld: Symbol not found: _ubrk_clone
  Referenced from: claude-darwin-x64 (which was built for Mac OS
X 13.0)
  Expected in: /usr/lib/libicucore.A.dylib
```

Solutions:

1. **Check your macOS version**: Claude Code requires macOS 13.0 or later. Open the Apple menu and select About This Mac to check your version.
2. **Update macOS** if you're on an older version. The binary uses load commands and system libraries that older macOS versions don't support. Alternative install methods like Homebrew download the same binary and won't resolve this error.

`Exec format error` on WSL1

If running `claude` in WSL prints `cannot execute binary file: Exec format error`, you're on WSL1 and hitting a known native-binary regression tracked in **issue #38788**. The binary's program headers changed in a way WSL1's loader can't handle.

The cleanest fix is to convert your distribution to WSL2 from PowerShell:

```
wsl --set-version <DistroName> 2
```

If you need to stay on WSL1, invoke the binary through the dynamic linker. Add this function to `~/.bashrc` inside WSL, replacing the path if your home directory differs:

```
claude() {
  /lib64/ld-linux-x86-64.so.2 "$(readlink -f
"$HOME/.local/bin/claude")" "$@"
}
```

Then run `source ~/.bashrc` and retry `claude`.

npm install errors in WSL

These issues apply if you installed Claude Code with `npm install -g` inside WSL. If you used the **native installer**, skip this section.

OS or platform detection issues. If npm reports a platform mismatch during install, WSL is likely picking up the Windows `npm`. Run `npm config set os linux` first, then install with `npm install -g @anthropic-ai/claude-code --force`. Do not use `sudo`.

`exec: node: not found` **when running** `claude`**.** Your WSL environment is likely using the Windows installation of Node.js. Confirm with `which npm` and `which node`: paths starting with `/mnt/c/` are Windows binaries, while Linux paths start with `/usr/`. To fix this, install Node via your Linux distribution's package manager or via `nvm`.

nvm version conflicts. If you have nvm installed in both WSL and Windows, switching Node versions in WSL may break because WSL imports the Windows PATH by default and the Windows nvm takes priority. The most common cause is that nvm isn't loaded in your shell. Add the nvm loader to `~/.bashrc` or `~/.zshrc`:

```
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
[ -s "$NVM_DIR/bash_completion" ] && \.
"$NVM_DIR/bash_completion"
```

Or load it in your current session:

```
source ~/.nvm/nvm.sh
```

If nvm is loaded but Windows paths still take priority, prepend your Linux Node path explicitly:

```
export PATH="$HOME/.nvm/versions/node/$(node -v)/bin:$PATH"
```

> ⚠ Avoid disabling Windows PATH importing via `appendWindowsPath = false` as this breaks the ability to call Windows executables from WSL. Similarly, avoid uninstalling Node.js from Windows if you use it for Windows development.

Permission errors during installation

If the native installer fails with permission errors, the target directory may not be writable. See **Check directory permissions**.

If you previously installed with npm and are hitting npm-specific permission errors, switch to the native installer:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Native binary not found after npm install

The `@anthropic-ai/claude-code` npm package pulls in the native binary through a per-platform optional dependency such as `@anthropic-ai/claude-code-darwin-arm64`. If running `claude` after install prints `Could not find native binary package "@anthropic-ai/claude-code-<platform>"`, check the following causes:

- **Optional dependencies are disabled.** Remove `--omit=optional` from your npm install command, `--no-optional` from pnpm, or `--ignore-optional` from yarn, and check that `.npmrc` does not set `optional=false`. Then reinstall. The native binary is delivered only as an optional dependency, so there is no JavaScript fallback if it is skipped.
- **Unsupported platform.** Prebuilt binaries are published for `darwin-arm64`, `darwin-x64`, `linux-x64`, `linux-arm64`, `linux-x64-musl`, `linux-arm64-musl`, `win32-x64`, and `win32-arm64`. Claude Code does not ship a binary for other platforms; see the **system requirements**.
- **Corporate npm mirror is missing the platform packages.** Ensure your registry mirrors all eight `@anthropic-ai/claude-code-*` platform packages in addition to the meta package.

Installing with `--ignore-scripts` does not trigger this error. The postinstall step that links the binary into place is skipped, so Claude Code falls back to a wrapper that locates and spawns the platform binary on each launch. This works but starts more slowly; reinstall with scripts enabled for

direct execution.

Login and authentication

These sections address login failures, OAuth errors, and token issues.

Reset your login

When login fails and the cause isn't obvious, a clean re-authentication resolves most cases:

1. Run `/logout` to sign out completely
2. Close Claude Code
3. Restart with `claude` and complete the authentication process again

If the browser doesn't open automatically during login, press `c` to copy the OAuth URL to your clipboard, then paste it into a browser manually. This also works when the URL wraps across lines in a narrow or SSH terminal and can't be clicked directly.

OAuth error: Invalid code

If you see `OAuth error: Invalid code. Please make sure the full code was copied`, the login code expired or was truncated during copy-paste.

Solutions:

- Press Enter to retry and complete the login quickly after the browser opens
- Type `c` to copy the full URL if the browser doesn't open automatically
- If using a remote/SSH session, the browser may open on the wrong machine. Copy the URL displayed in the terminal and open it in your local browser instead.

403 Forbidden after login

If you see `API Error: 403 {"error":{"type":"forbidden","message":"Request not allowed"}}` after logging in:

- **Claude Pro/Max users**: verify your subscription is active at **claude.ai/settings**
- **Anthropic Console users**: confirm your account has the "Claude Code" or "Developer" role.

Admins assign this in the Anthropic Console under Settings → Members.

- **Behind a proxy**: corporate proxies can interfere with API requests. See **network configuration** for proxy setup.

This organization has been disabled with an active subscription

If you see `API Error: 400 ... "This organization has been disabled"` despite having an active Claude subscription, an `ANTHROPIC_API_KEY` environment variable is overriding your subscription. This commonly happens when an old API key from a previous employer or project is still set in your shell profile.

When `ANTHROPIC_API_KEY` is present and you have approved it, Claude Code uses that key instead of your subscription's OAuth credentials. In non-interactive mode with the `-p` flag, the key is always used when present. See **authentication precedence** for the full resolution order.

To use your subscription instead, unset the environment variable and remove it from your shell profile:

```
unset ANTHROPIC_API_KEY
claude
```

Check `~/.zshrc`, `~/.bashrc`, or `~/.profile` for `export ANTHROPIC_API_KEY=...` lines and remove them to make the change permanent. On Windows, check your PowerShell profile at `$PROFILE` and your User environment variables for `ANTHROPIC_API_KEY`. Run `/status` inside Claude Code to confirm which authentication method is active.

OAuth login fails in WSL2, SSH, or containers

When Claude Code runs in WSL2, on a remote machine over SSH, or inside a container, the browser usually opens on a different host and its redirect can't reach Claude Code's local callback server. After you sign in, the browser shows a login code instead of redirecting back automatically. Paste that code into the terminal at the `Paste code here if prompted` prompt to complete login.

If the browser doesn't open at all from WSL2, set the `BROWSER` environment variable to your Windows browser path:

```
export BROWSER="/mnt/c/Program
Files/Google/Chrome/Application/chrome.exe"
claude
```

Alternatively, press `c` at the interactive login prompt to copy the OAuth URL, or copy the URL that `claude auth login` prints, and open it in a browser on your local machine.

If pasting the code into the interactive prompt does nothing, your terminal's paste binding likely isn't reaching the input field. Try your terminal's alternate paste shortcut, often right-click or Shift+Insert in Windows Terminal, or use `claude auth login` instead, which reads the pasted code from standard input:

```
claude auth login
```

This fallback also applies on native Windows or any terminal where pasting into the interactive prompt fails.

Not logged in or token expired

If Claude Code prompts you to log in again after a session, your OAuth token may have expired.

Run `/login` to re-authenticate. If this happens frequently, check that your system clock is accurate, as token validation depends on correct timestamps.

On macOS, login can also fail when the Keychain is locked or its password is out of sync with your account password, which prevents Claude Code from saving credentials. Run `claude doctor` to check Keychain access. To unlock the Keychain manually, run `security unlock-keychain ~/Library/Keychains/login.keychain-db`. If unlocking doesn't help, open Keychain Access, select the `login` keychain, and choose Edit > Change Password for Keychain "login" to resync it with your account password.

Bedrock, Vertex, or Foundry credentials not loading

If you configured Claude Code to use a cloud provider and see `Could not load credentials from any providers` on Bedrock, `Could not load the default credentials` on Vertex, or `ChainedTokenCredential authentication failed` on Foundry, your cloud provider CLI is likely not authenticated in the current shell.

For Bedrock, confirm your AWS credentials are valid:

```
aws sts get-caller-identity
```

For Vertex AI, confirm `ANTHROPIC_VERTEX_PROJECT_ID` and `CLOUD_ML_REGION` are set in your shell, then set application default credentials:

```
gcloud auth application-default login
```

For Microsoft Foundry, confirm `ANTHROPIC_FOUNDRY_API_KEY` is set, or sign in with the Azure CLI so the default credential chain can find your account:

```
az login
```

If credentials work in your terminal but not in the VS Code or JetBrains extension, the IDE process likely didn't inherit your shell environment. Set the provider environment variables in the IDE's own settings, or launch the IDE from a terminal where they're already exported.

See **Amazon Bedrock**, **Google Vertex AI**, or **Microsoft Foundry** for full provider setup.

Still stuck

If none of the above resolves your issue:

1. Check the **GitHub repository** for known issues, or open a new one with your operating system, the install command you ran, and the full error output
2. If `claude --version` works but something else is wrong, run `claude doctor` for an automated diagnostic report
3. If you can start a session, use `/feedback` inside Claude Code to report the problem